

```
In [50]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import random
```

NumPy Basics

```
In [87]: # 1. Create a NumPy array of numbers from 1 to 10 and print it.
a = np.arange(1,11)

print(a)
```

```
[ 1  2  3  4  5  6  7  8  9 10]
```

```
In [88]: # 2. Create 3x3 matrix filled with 0 and another filled with
b = np.zeros((3,3))
print(b)
b = np.ones((3,3))
print(b)
```

```
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
```

```
In [53]: # 3. Generate an array of 5 random numbers between 0 and 1.

random_array=np.random.rand(5) #random.rand is specifically designed to pick random

print(random_array)
```

```
[0.57112786 0.91594619 0.93990941 0.75315575 0.36174379]
```

```
In [54]: # 4. Find the sum, mean, minimum, and maximum of a NumPy array.

a = np.array([1,2,3,4,5])
sum = np.sum(a)
mean = np.mean(a)
minimum = np.min(a)
max = np.max(a)

print(f"Sum is {sum}")
print(f"Mean is {mean}")
print(f"Minimum is {sum}")
print(f"Sum is {sum}")
```

```
Sum is 15
Mean is 3.0
Minimum is 15
Sum is 15
```

```
In [55]: # 5. Reshape a 1D array of 12 elements into a 3x4 matrix.
```

```
a = np.array([2,3,4,7,5,6,7,7,7,7,10,200]).reshape(3,4)
a
```

```
Out[55]: array([[ 2,  3,  4,  7],
               [ 5,  6,  7,  7],
               [ 7,  7, 10, 200]])
```

```
In [56]: # 6. Multiplying 2 arrays index-wise and print the result.
```

```
a = np.array([10,20,30,40,50])
b = np.array([2,3,4,10,5])
c = a * b
c
```

```
Out[56]: array([ 20,  60, 120, 400, 250])
```

```
In [57]: # 7. Slice the first 3 elements and the last 3 elements from an array.
```

```
a = np.array([10,23,56,43,45,623,47,38,29,10,11,234,43,1])
print(a[:3])
print(a[11:])
```

```
[10 23 56]
[234 43  1]
```

```
In [58]: # 8. Create an identity matrix of size 4x4 using NumPy.
```

```
print(np.identity(4))
```

```
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

```
In [ ]:
```

Pandas Basics

```
In [59]: # 1. Create a Pandas DataFrame with columns: Name, Age, City, and Salary.
```

```
data = {
    "Name": ['Saugat Tamang', 'augat Amang', 'Ugat mang',
            'Gat Ang'],
    "Age": [19, 20, None, 22],
    "City": ['Kathmandu', 'Plutonium', 'Kebab City',
            None],
    "Salary": [300, 400, 1000000000, 1000000]
}

df = pd.DataFrame(data)
df
```

Out[59]:

	Name	Age	City	Salary
0	Saugat Tamang	19.0	Kathmandu	300
1	augat Amang	20.0	Plutonium	400
2	Ugat mang	NaN	Kebab City	1000000000
3	Gat Ang	22.0	NaN	1000000

In [60]: *# 2. Display the first 5 rows and last 3 rows of a DataFrame.*

```
df.head(5)
df.tail(3)
```

Out[60]:

	Name	Age	City	Salary
1	augat Amang	20.0	Plutonium	400
2	Ugat mang	NaN	Kebab City	1000000000
3	Gat Ang	22.0	NaN	1000000

In [61]: *# 3. Check the shape, data types, and basic statistics of a DataFrame using info()*

```
df.info()
df.describe()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 4 entries, 0 to 3
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Name      4 non-null      str
1    Age        3 non-null      float64
2    City       3 non-null      str
3    Salary     4 non-null      int64
dtypes: float64(1), int64(1), str(2)
memory usage: 260.0 bytes
```

Out[61]:

	Age	Salary
count	3.000000	4.000000e+00
mean	20.333333	2.502502e+08
std	1.527525	4.998334e+08
min	19.000000	3.000000e+02
25%	19.500000	3.750000e+02
50%	20.000000	5.002000e+05
75%	21.000000	2.507500e+08
max	22.000000	1.000000e+09

In [62]: *# 4. Select a single column and multiple columns from a DataFrame.*

```
a = df[['Name']]
b = df[['Name', 'Age', 'City', 'Salary']]
print(a)
print(b)
```

```
      Name
0  Saugat Tamang
1   augat Amang
2    Ugat mang
3     Gat Ang
      Name  Age  City  Salary
0  Saugat Tamang  19.0  Kathmandu    300
1   augat Amang  20.0  Plutonium    400
2    Ugat mang   NaN  Kebab City 1000000000
3     Gat Ang  22.0     NaN    1000000
```

In [63]: *# 5. Filter rows where Salary is greater than 40000.*

```
high_salary = df[df['Salary'] > 40000]
print(high_salary)
```

```
      Name  Age  City  Salary
2  Ugat mang   NaN  Kebab City 1000000000
3   Gat Ang  22.0     NaN    1000000
```

In [64]: *# 6. Add a new column 'Tax' which is 10% of Salary.*

```
Tax = df['Salary'] * 0.1
df['Tax'] = Tax
df
```

Out[64]:

	Name	Age	City	Salary	Tax
0	Saugat Tamang	19.0	Kathmandu	300	30.0
1	augat Amang	20.0	Plutonium	400	40.0
2	Ugat mang	NaN	Kebab City	1000000000	100000000.0
3	Gat Ang	22.0	NaN	1000000	100000.0

In [65]: *# 7. Sort the DataFrame by Age in ascending order.*

```
df_sorted = df.sort_values(by='Age')
df_sorted
```

Out[65]:

	Name	Age	City	Salary	Tax
0	Saugat Tamang	19.0	Kathmandu	300	30.0
1	augat Amang	20.0	Plutonium	400	40.0
3	Gat Ang	22.0	NaN	1000000	100000.0
2	Ugat mang	NaN	Kebab City	1000000000	100000000.0

In [66]: *# 8. Find and count missing values in each column using isnull().sum()*

```
df.isnull().sum()
```

```
Out[66]: Name      0
Age        1
City       1
Salary     0
Tax        0
dtype: int64
```

In [67]: *# 9. Replace missing values in a numeric column with the column mean.*

```
df.loc[2, 'Salary'] = np.nan
df.loc[3, 'Salary'] = np.nan
print(df)

df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df
```

	Name	Age	City	Salary	Tax
0	Saugat Tamang	19.0	Kathmandu	300.0	30.0
1	augat Amang	20.0	Plutonium	400.0	40.0
2	Ugat mang	NaN	Kebab City	NaN	100000000.0
3	Gat Ang	22.0	NaN	NaN	100000.0

Out[67]:

	Name	Age	City	Salary	Tax
0	Saugat Tamang	19.0	Kathmandu	300.0	30.0
1	augat Amang	20.0	Plutonium	400.0	40.0
2	Ugat mang	NaN	Kebab City	350.0	100000000.0
3	Gat Ang	22.0	NaN	350.0	100000.0

In [68]: *# 10. Group data by City and find the average Salary per City.*

```
avg_salary_by_city = df.groupby('City')['Salary'].mean()
print(avg_salary_by_city)
```

```
City
Kathmandu    300.0
Kebab City   350.0
Plutonium    400.0
Name: Salary, dtype: float64
```

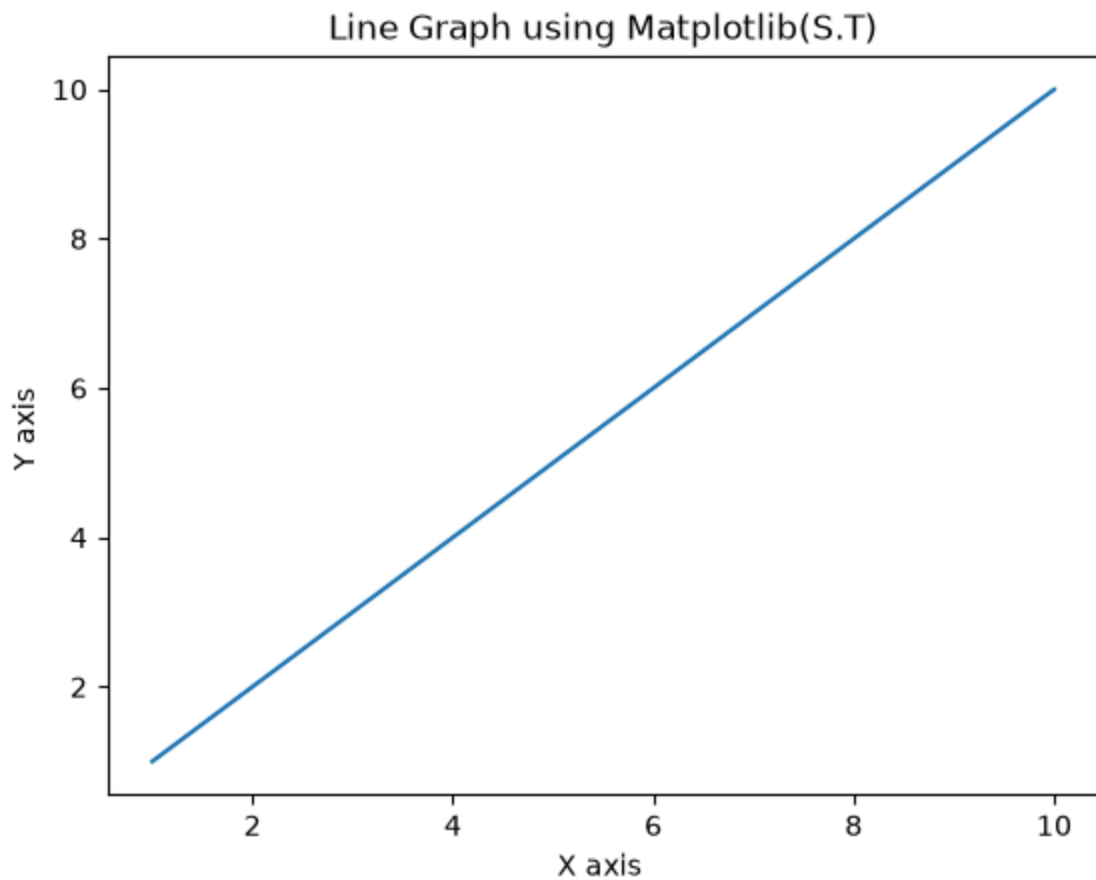
Matplotlib Visualization

In [69]: *# 1. Plot a simple Line graph showing numbers 1 to 10 on both axes.*

```
x = [1,2,3,4,5,6,7,8,9,10]
y = [1,2,3,4,5,6,7,8,9,10]

plt.plot(x,y)
plt.title("Line Graph using Matplotlib(S.T)")
plt.xlabel("X axis ")
plt.ylabel("Y axis ")
```

Out[69]: Text(0, 0.5, 'Y axis ')



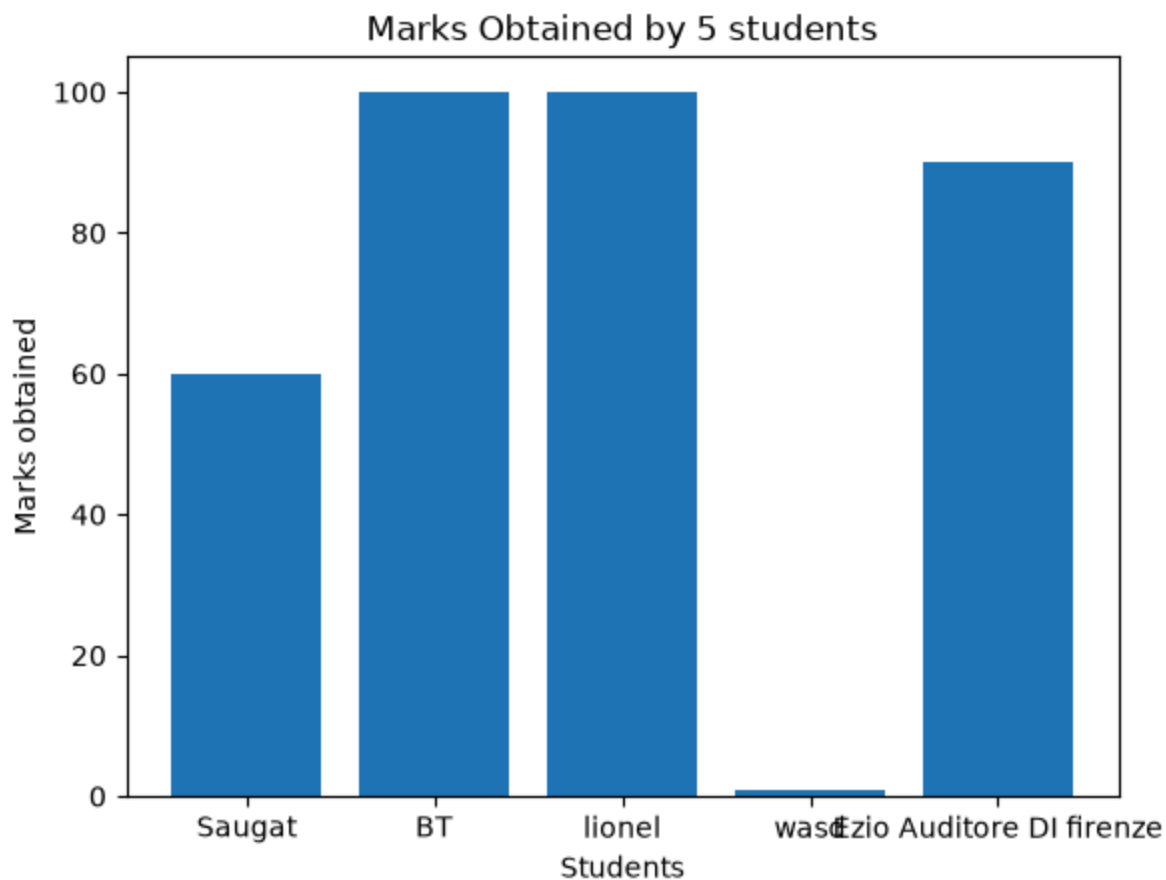
In [70]: *# 2. Create a bar chart showing marks of 5 students.*

```
students = ["Saugat", "BT", "lionel", "wasd", "Ezio Auditore DI firenze"]
marks = [60, 100, 100, 1, 90]

plt.bar(students, marks)

plt.title("Marks Obtained by 5 students")
plt.xlabel("Students")
plt.ylabel("Marks obtained")

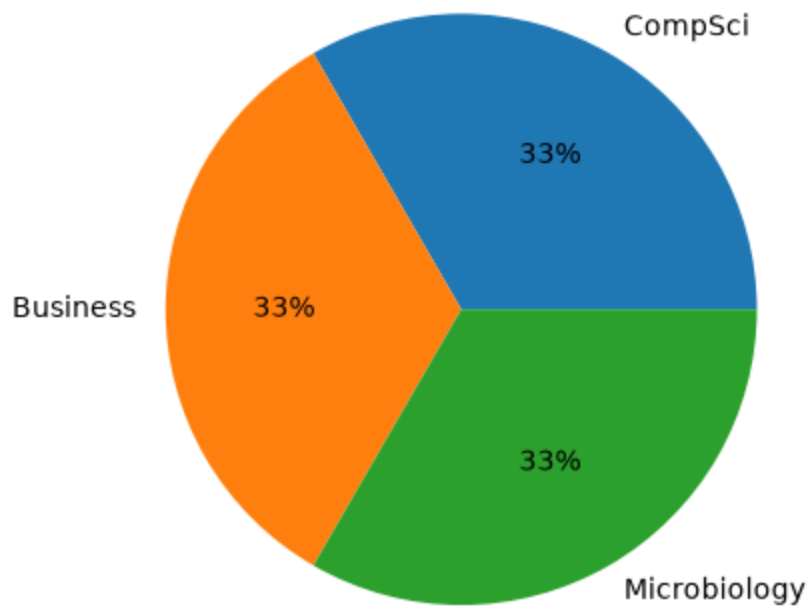
plt.show()
```



In [71]: *# 3. Draw a pie chart showing percentage of students in 3 departments.*

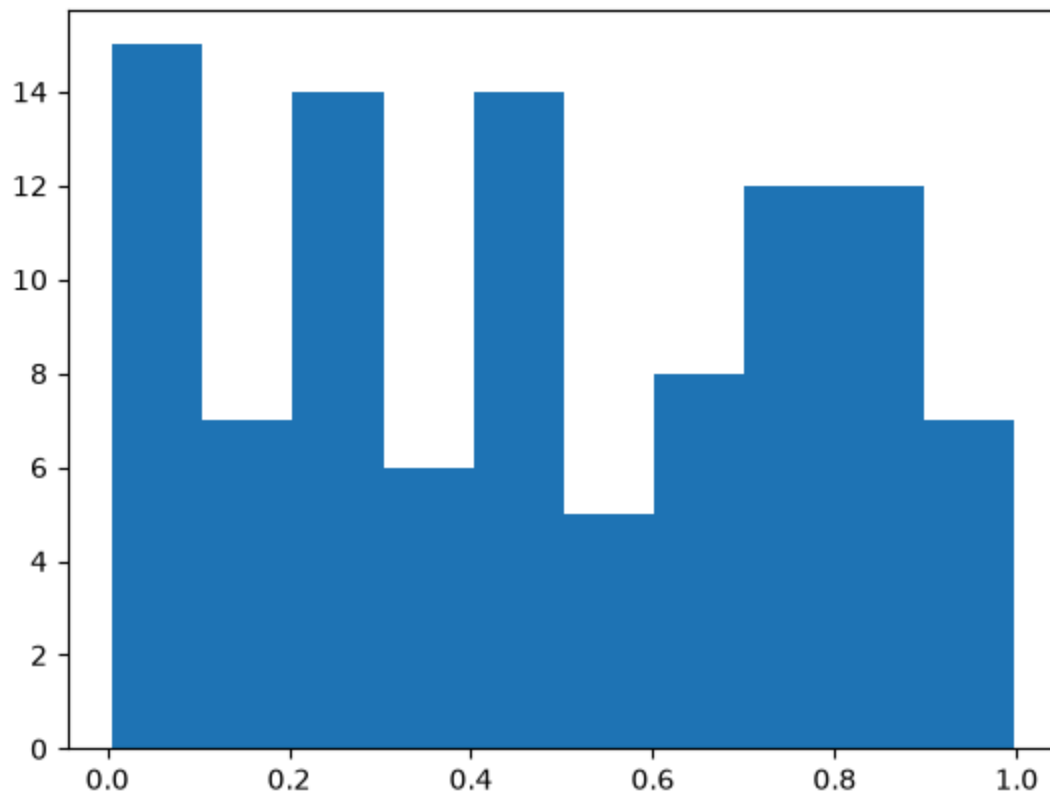
```
departments = ["CompSci", "Business", "Microbiology"]  
num_students = [444, 444, 444]  
  
plt.pie(num_students, labels=departments, autopct="%d%%")  
plt.title("Students in Each Department of college")  
plt.show()
```


Students in Each Department of college



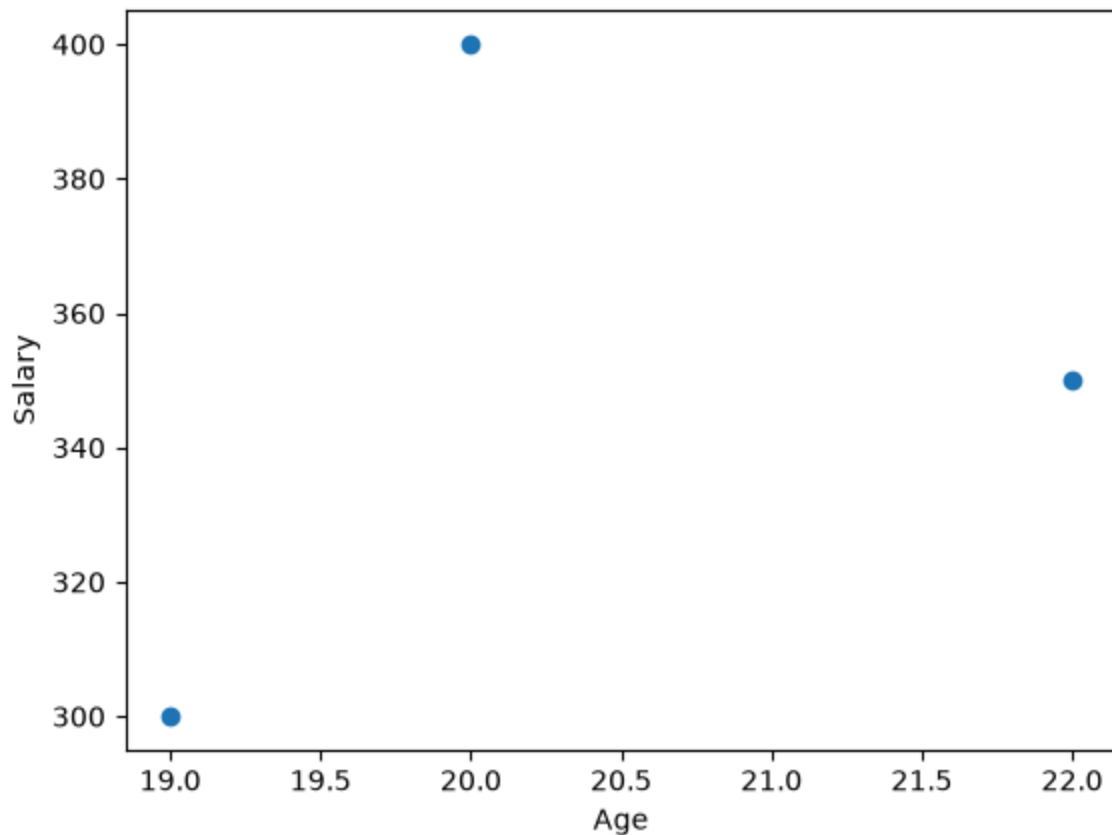
In [72]: *# 4. Plot a histogram for 100 random numbers to see the distribution.*

```
x = np.random.random(100)
plt.hist(x)
plt.show()
```



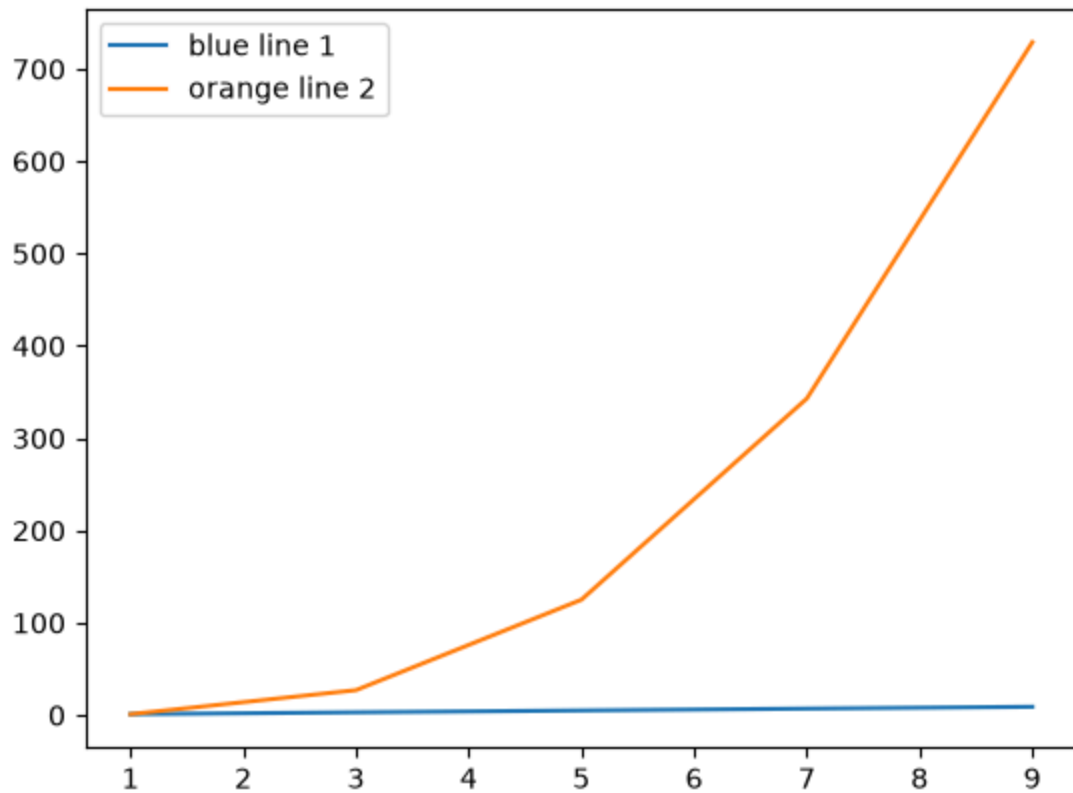
In [73]: *# 5. Create a scatter plot between Age and Salary from a sample dataset.*

```
plt.scatter(df['Age'], df['Salary'])  
plt.xlabel("Age")  
plt.ylabel("Salary")  
plt.show()
```



In [74]: *# 6. Plot two lines on the same graph and add a Legend.*

```
x = np.array([1, 3, 5, 7, 9])  
y1 = x  
y2 = x**3  
  
plt.plot(x, y1, label="blue line 1")  
plt.plot(x, y2, label="orange line 2")  
  
plt.legend()  
plt.show()
```

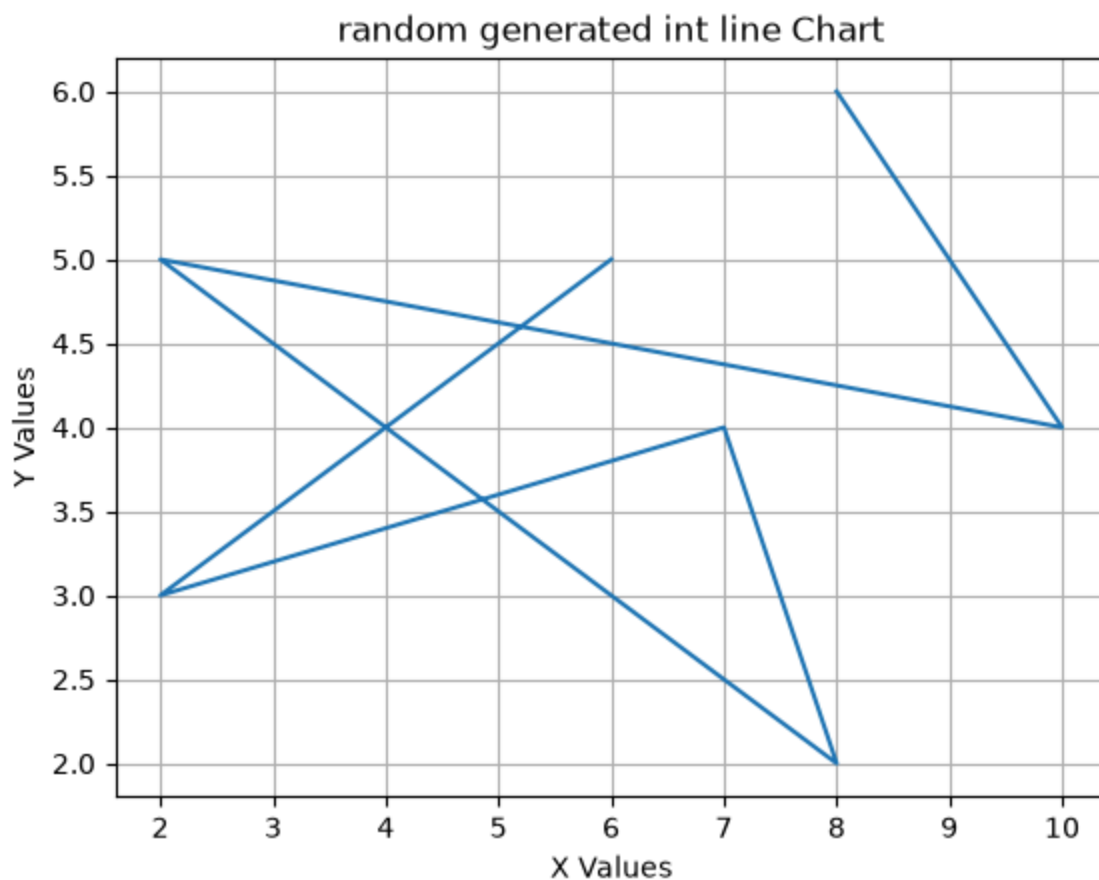


In [75]: # 7. Add title, x-label, y-label, and grid to any chart.

```
x=np.random.randint(1,11,size=7)
y=np.random.randint(1,11,size=7)

plt.plot(x, y)

plt.title("random generated int line Chart")
plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.grid(True)
plt.show()
```



In [76]: # 8. Use subplots to display a line chart and a bar chart side by side.

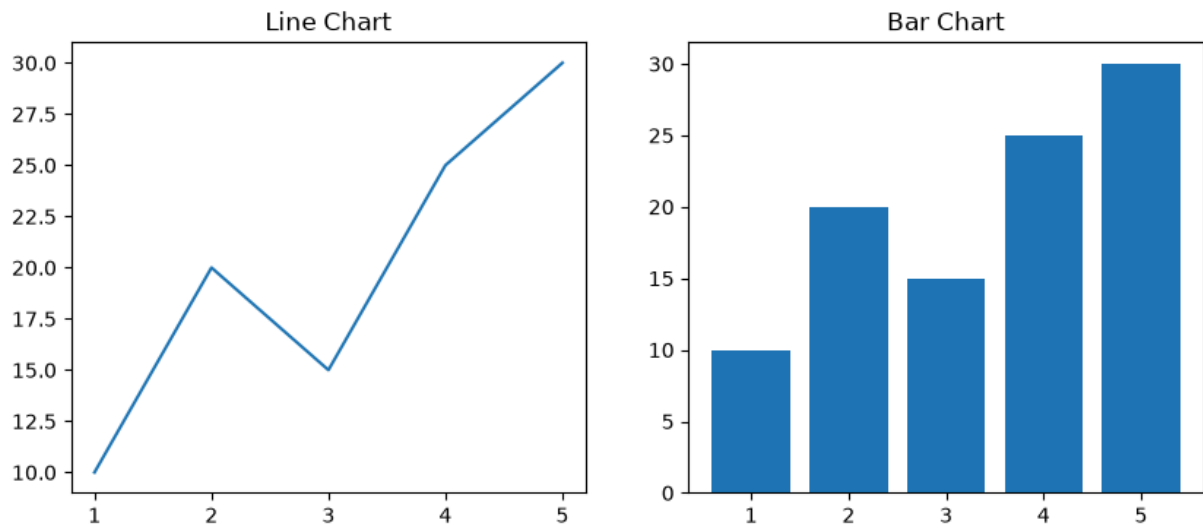
```
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.plot(x, y)
plt.title("Line Chart")

plt.subplot(1, 2, 2)
plt.bar(x, y)
plt.title("Bar Chart")

plt.show()
```



Reading & Cleaning CSV Files

In [77]: *# 1. Read a CSV file using pandas read_csv() and display its contents.*

```
df= pd.read_csv('data_breast_cancer.csv')
print(df.columns)
df = df.drop(columns=['id'])
df
```

```
Index(['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
      'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
      'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
      'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
      'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
      'fractal_dimension_se', 'radius_worst', 'texture_worst',
      'perimeter_worst', 'area_worst', 'smoothness_worst',
      'compactness_worst', 'concavity_worst', 'concave points_worst',
      'symmetry_worst', 'fractal_dimension_worst', 'Unnamed: 32'],
      dtype='str')
```

Out[77]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mear
0	M	17.99	10.38	122.80	1001.0	0.11840
1	M	20.57	17.77	132.90	1326.0	0.08474
2	M	19.69	21.25	130.00	1203.0	0.10960
3	M	11.42	20.38	77.58	386.1	0.14250
4	M	20.29	14.34	135.10	1297.0	0.10030
...	...	...	...	...	...	...
564	M	21.56	22.39	142.00	1479.0	0.11100
565	M	20.13	28.25	131.20	1261.0	0.09780
566	M	16.60	28.08	108.30	858.1	0.08455
567	M	20.60	29.33	140.10	1265.0	0.11780
568	B	7.76	24.54	47.92	181.0	0.05263

569 rows × 32 columns



In [78]:

```
# 2. Print the column names, number of rows, and number of columns.  
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 32 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   diagnosis                             569 non-null    str
1   radius_mean                           569 non-null    float64
2   texture_mean                           569 non-null    float64
3   perimeter_mean                         569 non-null    float64
4   area_mean                             569 non-null    float64
5   smoothness_mean                       569 non-null    float64
6   compactness_mean                      569 non-null    float64
7   concavity_mean                        569 non-null    float64
8   concave points_mean                   569 non-null    float64
9   symmetry_mean                         569 non-null    float64
10  fractal_dimension_mean                569 non-null    float64
11  radius_se                             569 non-null    float64
12  texture_se                             569 non-null    float64
13  perimeter_se                           569 non-null    float64
14  area_se                               569 non-null    float64
15  smoothness_se                         569 non-null    float64
16  compactness_se                        569 non-null    float64
17  concavity_se                          569 non-null    float64
18  concave points_se                     569 non-null    float64
19  symmetry_se                           569 non-null    float64
20  fractal_dimension_se                  569 non-null    float64
21  radius_worst                          569 non-null    float64
22  texture_worst                         569 non-null    float64
23  perimeter_worst                       569 non-null    float64
24  area_worst                            569 non-null    float64
25  smoothness_worst                      569 non-null    float64
26  compactness_worst                     569 non-null    float64
27  concavity_worst                       569 non-null    float64
28  concave points_worst                  569 non-null    float64
29  symmetry_worst                        569 non-null    float64
30  fractal_dimension_worst                569 non-null    float64
31  Unnamed: 32                           0 non-null      float64
dtypes: float64(31), str(1)
memory usage: 142.4 KB

```

In [79]: *# 3. Display the first 5 rows and check for missing values.*

```

print(df.head(5))
print(df.isnull().sum())

```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	\
0	M	17.99	10.38	122.80	1001.0	
1	M	20.57	17.77	132.90	1326.0	
2	M	19.69	21.25	130.00	1203.0	
3	M	11.42	20.38	77.58	386.1	
4	M	20.29	14.34	135.10	1297.0	

	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	\
0	0.11840	0.27760	0.3001	0.14710	
1	0.08474	0.07864	0.0869	0.07017	
2	0.10960	0.15990	0.1974	0.12790	
3	0.14250	0.28390	0.2414	0.10520	
4	0.10030	0.13280	0.1980	0.10430	

	symmetry_mean	...	texture_worst	perimeter_worst	area_worst	\
0	0.2419	...	17.33	184.60	2019.0	
1	0.1812	...	23.41	158.80	1956.0	
2	0.2069	...	25.53	152.50	1709.0	
3	0.2597	...	26.50	98.87	567.7	
4	0.1809	...	16.67	152.20	1575.0	

	smoothness_worst	compactness_worst	concavity_worst	concave points_worst	\
0	0.1622	0.6656	0.7119	0.2654	
1	0.1238	0.1866	0.2416	0.1860	
2	0.1444	0.4245	0.4504	0.2430	
3	0.2098	0.8663	0.6869	0.2575	
4	0.1374	0.2050	0.4000	0.1625	

	symmetry_worst	fractal_dimension_worst	Unnamed: 32
0	0.4601	0.11890	NaN
1	0.2750	0.08902	NaN
2	0.3613	0.08758	NaN
3	0.6638	0.17300	NaN
4	0.2364	0.07678	NaN

[5 rows x 32 columns]

diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0
perimeter_se	0
area_se	0
smoothness_se	0
compactness_se	0
concavity_se	0
concave points_se	0
symmetry_se	0


```

fractal_dimension_se      0
radius_worst              0
texture_worst             0
perimeter_worst          0
area_worst                0
smoothness_worst         0
compactness_worst        0
concavity_worst          0
concave points_worst     0
symmetry_worst           0
fractal_dimension_worst   0
Unnamed: 32              569
dtype: int64

```

In [80]: *# 4. Drop rows that contain any missing values using dropna().*

```

df_cleaned = df.dropna()
df_cleaned

```

Out[80]:

diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	c
-----------	-------------	--------------	----------------	-----------	-----------------	---

0 rows × 32 columns



In [81]: *# 5. Fill missing numeric values with the column mean using fillna()*

```

numeric_cols = [
    'radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
    'smoothness_mean', 'compactness_mean', 'concavity_mean',
    'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
    'radius_se', 'texture_se', 'perimeter_se', 'area_se',
    'smoothness_se', 'compactness_se', 'concavity_se',
    'concave points_se', 'symmetry_se', 'fractal_dimension_se',
    'radius_worst', 'texture_worst', 'perimeter_worst',
    'area_worst', 'smoothness_worst', 'compactness_worst',
    'concavity_worst', 'concave points_worst',
    'symmetry_worst', 'fractal_dimension_worst'
]

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

# verify there are no missing values left
print(df.isnull().sum())

```

```

diagnosis      0
radius_mean    0
texture_mean    0
perimeter_mean 0
area_mean      0
smoothness_mean 0
compactness_mean 0
concavity_mean 0
concave points_mean 0
symmetry_mean  0
fractal_dimension_mean 0
radius_se      0
texture_se     0
perimeter_se   0
area_se        0
smoothness_se  0
compactness_se 0
concavity_se   0
concave points_se 0
symmetry_se    0
fractal_dimension_se 0
radius_worst   0
texture_worst  0
perimeter_worst 0
area_worst     0
smoothness_worst 0
compactness_worst 0
concavity_worst 0
concave points_worst 0
symmetry_worst 0
fractal_dimension_worst 0
Unnamed: 32    569
dtype: int64

```

```

In [82]: # 6. Remove duplicate rows from the dataset.
print(df.duplicated().sum())
df = df.drop_duplicates()

# optional: reset the index after dropping rows
df = df.reset_index(drop=True)

print(df.shape)

```

```

0
(569, 32)

```

```

In [83]: # 7. Rename column headers to lowercase using str.lower().

df.columns = df.columns.str.lower()
print(df.columns)

```

```
Index(['diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
      'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
      'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
      'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
      'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
      'fractal_dimension_se', 'radius_worst', 'texture_worst',
      'perimeter_worst', 'area_worst', 'smoothness_worst',
      'compactness_worst', 'concavity_worst', 'concave points_worst',
      'symmetry_worst', 'fractal_dimension_worst', 'unnamed: 32'],
      dtype='str')
```

In [84]: *# 8. Convert a categorical column (e.g. Gender: Male/Female) into numbers using map*

```
df["diagnosis"] = df["diagnosis"].replace({
    "B": 0,
    "M": 1
})

df
```

Out[84]:

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean
0	1	17.99	10.38	122.80	1001.0	0.11840
1	1	20.57	17.77	132.90	1326.0	0.08474
2	1	19.69	21.25	130.00	1203.0	0.10960
3	1	11.42	20.38	77.58	386.1	0.14250
4	1	20.29	14.34	135.10	1297.0	0.10030
...	...	...	...	...	...	...
564	1	21.56	22.39	142.00	1479.0	0.11100
565	1	20.13	28.25	131.20	1261.0	0.09780
566	1	16.60	28.08	108.30	858.1	0.08455
567	1	20.60	29.33	140.10	1265.0	0.11780
568	0	7.76	24.54	47.92	181.0	0.05263

569 rows × 7 columns



In [85]: *# 9. Select only numeric columns and print their correlation using corr().*

```
print(df.select_dtypes(include=['number']).corr())
```

	radius_mean	texture_mean	perimeter_mean	area_mean	\
radius_mean	1.000000	0.323782	0.997855	0.987357	
texture_mean	0.323782	1.000000	0.329533	0.321086	
perimeter_mean	0.997855	0.329533	1.000000	0.986507	
area_mean	0.987357	0.321086	0.986507	1.000000	
smoothness_mean	0.170581	-0.023389	0.207278	0.177028	
compactness_mean	0.506124	0.236702	0.556936	0.498502	
concavity_mean	0.676764	0.302418	0.716136	0.685983	
concave points_mean	0.822529	0.293464	0.850977	0.823269	
symmetry_mean	0.147741	0.071401	0.183027	0.151293	
fractal_dimension_mean	-0.311631	-0.076437	-0.261477	-0.283110	
radius_se	0.679090	0.275869	0.691765	0.732562	
texture_se	-0.097317	0.386358	-0.086761	-0.066280	
perimeter_se	0.674172	0.281673	0.693135	0.726628	
area_se	0.735864	0.259845	0.744983	0.800086	
smoothness_se	-0.222600	0.006614	-0.202694	-0.166777	
compactness_se	0.206000	0.191975	0.250744	0.212583	
concavity_se	0.194204	0.143293	0.228082	0.207660	
concave points_se	0.376169	0.163851	0.407217	0.372320	
symmetry_se	-0.104321	0.009127	-0.081629	-0.072497	
fractal_dimension_se	-0.042641	0.054458	-0.005523	-0.019887	
radius_worst	0.969539	0.352573	0.969476	0.962746	
texture_worst	0.297008	0.912045	0.303038	0.287489	
perimeter_worst	0.965137	0.358040	0.970387	0.959120	
area_worst	0.941082	0.343546	0.941550	0.959213	
smoothness_worst	0.119616	0.077503	0.150549	0.123523	
compactness_worst	0.413463	0.277830	0.455774	0.390410	
concavity_worst	0.526911	0.301025	0.563879	0.512606	
concave points_worst	0.744214	0.295316	0.771241	0.722017	
symmetry_worst	0.163953	0.105008	0.189115	0.143570	
fractal_dimension_worst	0.007066	0.119205	0.051019	0.003738	
unnamed: 32	NaN	NaN	NaN	NaN	

	smoothness_mean	compactness_mean	concavity_mean	\
radius_mean	0.170581	0.506124	0.676764	
texture_mean	-0.023389	0.236702	0.302418	
perimeter_mean	0.207278	0.556936	0.716136	
area_mean	0.177028	0.498502	0.685983	
smoothness_mean	1.000000	0.659123	0.521984	
compactness_mean	0.659123	1.000000	0.883121	
concavity_mean	0.521984	0.883121	1.000000	
concave points_mean	0.553695	0.831135	0.921391	
symmetry_mean	0.557775	0.602641	0.500667	
fractal_dimension_mean	0.584792	0.565369	0.336783	
radius_se	0.301467	0.497473	0.631925	
texture_se	0.068406	0.046205	0.076218	
perimeter_se	0.296092	0.548905	0.660391	
area_se	0.246552	0.455653	0.617427	
smoothness_se	0.332375	0.135299	0.098564	
compactness_se	0.318943	0.738722	0.670279	
concavity_se	0.248396	0.570517	0.691270	
concave points_se	0.380676	0.642262	0.683260	
symmetry_se	0.200774	0.229977	0.178009	
fractal_dimension_se	0.283607	0.507318	0.449301	
radius_worst	0.213120	0.535315	0.688236	
texture_worst	0.036072	0.248133	0.299879	

perimeter_worst	0.238853	0.590210	0.729565
area_worst	0.206718	0.509604	0.675987
smoothness_worst	0.805324	0.565541	0.448822
compactness_worst	0.472468	0.865809	0.754968
concavity_worst	0.434926	0.816275	0.884103
concave points_worst	0.503053	0.815573	0.861323
symmetry_worst	0.394309	0.510223	0.409464
fractal_dimension_worst	0.499316	0.687382	0.514930
unnamed: 32	NaN	NaN	NaN

	concave points_mean	symmetry_mean \
radius_mean	0.822529	0.147741
texture_mean	0.293464	0.071401
perimeter_mean	0.850977	0.183027
area_mean	0.823269	0.151293
smoothness_mean	0.553695	0.557775
compactness_mean	0.831135	0.602641
concavity_mean	0.921391	0.500667
concave points_mean	1.000000	0.462497
symmetry_mean	0.462497	1.000000
fractal_dimension_mean	0.166917	0.479921
radius_se	0.698050	0.303379
texture_se	0.021480	0.128053
perimeter_se	0.710650	0.313893
area_se	0.690299	0.223970
smoothness_se	0.027653	0.187321
compactness_se	0.490424	0.421659
concavity_se	0.439167	0.342627
concave points_se	0.615634	0.393298
symmetry_se	0.095351	0.449137
fractal_dimension_se	0.257584	0.331786
radius_worst	0.830318	0.185728
texture_worst	0.292752	0.090651
perimeter_worst	0.855923	0.219169
area_worst	0.809630	0.177193
smoothness_worst	0.452753	0.426675
compactness_worst	0.667454	0.473200
concavity_worst	0.752399	0.433721
concave points_worst	0.910155	0.430297
symmetry_worst	0.375744	0.699826
fractal_dimension_worst	0.368661	0.438413
unnamed: 32	NaN	NaN

	fractal_dimension_mean	...	texture_worst \
radius_mean	-0.311631	...	0.297008
texture_mean	-0.076437	...	0.912045
perimeter_mean	-0.261477	...	0.303038
area_mean	-0.283110	...	0.287489
smoothness_mean	0.584792	...	0.036072
compactness_mean	0.565369	...	0.248133
concavity_mean	0.336783	...	0.299879
concave points_mean	0.166917	...	0.292752
symmetry_mean	0.479921	...	0.090651
fractal_dimension_mean	1.000000	...	-0.051269
radius_se	0.000111	...	0.194799
texture_se	0.164174	...	0.409003

perimeter_se	0.039830	...	0.200371
area_se	-0.090170	...	0.196497
smoothness_se	0.401964	...	-0.074743
compactness_se	0.559837	...	0.143003
concavity_se	0.446630	...	0.100241
concave points_se	0.341198	...	0.086741
symmetry_se	0.345007	...	-0.077473
fractal_dimension_se	0.688132	...	-0.003195
radius_worst	-0.253691	...	0.359921
texture_worst	-0.051269	...	1.000000
perimeter_worst	-0.205151	...	0.365098
area_worst	-0.231854	...	0.345842
smoothness_worst	0.504942	...	0.225429
compactness_worst	0.458798	...	0.360832
concavity_worst	0.346234	...	0.368366
concave points_worst	0.175325	...	0.359755
symmetry_worst	0.334019	...	0.233027
fractal_dimension_worst	0.767297	...	0.219122
unnamed: 32	NaN	...	NaN

	perimeter_worst	area_worst	smoothness_worst	\
radius_mean	0.965137	0.941082	0.119616	
texture_mean	0.358040	0.343546	0.077503	
perimeter_mean	0.970387	0.941550	0.150549	
area_mean	0.959120	0.959213	0.123523	
smoothness_mean	0.238853	0.206718	0.805324	
compactness_mean	0.590210	0.509604	0.565541	
concavity_mean	0.729565	0.675987	0.448822	
concave points_mean	0.855923	0.809630	0.452753	
symmetry_mean	0.219169	0.177193	0.426675	
fractal_dimension_mean	-0.205151	-0.231854	0.504942	
radius_se	0.719684	0.751548	0.141919	
texture_se	-0.102242	-0.083195	-0.073658	
perimeter_se	0.721031	0.730713	0.130054	
area_se	0.761213	0.811408	0.125389	
smoothness_se	-0.217304	-0.182195	0.314457	
compactness_se	0.260516	0.199371	0.227394	
concavity_se	0.226680	0.188353	0.168481	
concave points_se	0.394999	0.342271	0.215351	
symmetry_se	-0.103753	-0.110343	-0.012662	
fractal_dimension_se	-0.001000	-0.022736	0.170568	
radius_worst	0.993708	0.984015	0.216574	
texture_worst	0.365098	0.345842	0.225429	
perimeter_worst	1.000000	0.977578	0.236775	
area_worst	0.977578	1.000000	0.209145	
smoothness_worst	0.236775	0.209145	1.000000	
compactness_worst	0.529408	0.438296	0.568187	
concavity_worst	0.618344	0.543331	0.518523	
concave points_worst	0.816322	0.747419	0.547691	
symmetry_worst	0.269493	0.209146	0.493838	
fractal_dimension_worst	0.138957	0.079647	0.617624	
unnamed: 32	NaN	NaN	NaN	

	compactness_worst	concavity_worst	\
radius_mean	0.413463	0.526911	
texture_mean	0.277830	0.301025	

perimeter_mean	0.455774	0.563879
area_mean	0.390410	0.512606
smoothness_mean	0.472468	0.434926
compactness_mean	0.865809	0.816275
concavity_mean	0.754968	0.884103
concave points_mean	0.667454	0.752399
symmetry_mean	0.473200	0.433721
fractal_dimension_mean	0.458798	0.346234
radius_se	0.287103	0.380585
texture_se	-0.092439	-0.068956
perimeter_se	0.341919	0.418899
area_se	0.283257	0.385100
smoothness_se	-0.055558	-0.058298
compactness_se	0.678780	0.639147
concavity_se	0.484858	0.662564
concave points_se	0.452888	0.549592
symmetry_se	0.060255	0.037119
fractal_dimension_se	0.390159	0.379975
radius_worst	0.475820	0.573975
texture_worst	0.360832	0.368366
perimeter_worst	0.529408	0.618344
area_worst	0.438296	0.543331
smoothness_worst	0.568187	0.518523
compactness_worst	1.000000	0.892261
concavity_worst	0.892261	1.000000
concave points_worst	0.801080	0.855434
symmetry_worst	0.614441	0.532520
fractal_dimension_worst	0.810455	0.686511
unnamed: 32	NaN	NaN

	concave points_worst	symmetry_worst \
radius_mean	0.744214	0.163953
texture_mean	0.295316	0.105008
perimeter_mean	0.771241	0.189115
area_mean	0.722017	0.143570
smoothness_mean	0.503053	0.394309
compactness_mean	0.815573	0.510223
concavity_mean	0.861323	0.409464
concave points_mean	0.910155	0.375744
symmetry_mean	0.430297	0.699826
fractal_dimension_mean	0.175325	0.334019
radius_se	0.531062	0.094543
texture_se	-0.119638	-0.128215
perimeter_se	0.554897	0.109930
area_se	0.538166	0.074126
smoothness_se	-0.102007	-0.107342
compactness_se	0.483208	0.277878
concavity_se	0.440472	0.197788
concave points_se	0.602450	0.143116
symmetry_se	-0.030413	0.389402
fractal_dimension_se	0.215204	0.111094
radius_worst	0.787424	0.243529
texture_worst	0.359755	0.233027
perimeter_worst	0.816322	0.269493
area_worst	0.747419	0.209146
smoothness_worst	0.547691	0.493838

compactness_worst	0.801080	0.614441
concavity_worst	0.855434	0.532520
concave points_worst	1.000000	0.502528
symmetry_worst	0.502528	1.000000
fractal_dimension_worst	0.511114	0.537848
unnamed: 32	NaN	NaN

	fractal_dimension_worst	unnamed: 32
radius_mean	0.007066	NaN
texture_mean	0.119205	NaN
perimeter_mean	0.051019	NaN
area_mean	0.003738	NaN
smoothness_mean	0.499316	NaN
compactness_mean	0.687382	NaN
concavity_mean	0.514930	NaN
concave points_mean	0.368661	NaN
symmetry_mean	0.438413	NaN
fractal_dimension_mean	0.767297	NaN
radius_se	0.049559	NaN
texture_se	-0.045655	NaN
perimeter_se	0.085433	NaN
area_se	0.017539	NaN
smoothness_se	0.101480	NaN
compactness_se	0.590973	NaN
concavity_se	0.439329	NaN
concave points_se	0.310655	NaN
symmetry_se	0.078079	NaN
fractal_dimension_se	0.591328	NaN
radius_worst	0.093492	NaN
texture_worst	0.219122	NaN
perimeter_worst	0.138957	NaN
area_worst	0.079647	NaN
smoothness_worst	0.617624	NaN
compactness_worst	0.810455	NaN
concavity_worst	0.686511	NaN
concave points_worst	0.511114	NaN
symmetry_worst	0.537848	NaN
fractal_dimension_worst	1.000000	NaN
unnamed: 32	NaN	NaN

[31 rows x 31 columns]

In [86]: *# 10. Save the cleaned DataFrame to a new CSV file using to_csv().*

```
df.to_csv('cleaned_data_breast_cancer.csv', index=False)

print("DataFrame successfully saved as 'cleaned_data_breast_cancer.csv'")
```

DataFrame successfully saved as 'cleaned_data_breast_cancer.csv'

In []:

In []: